Actividad de Aprendizaje

Ejercicios propuestos

Ejercicio 1: Patrón Strategy (planilla de sueldos)

Una empresa paga a sus empleados bajo tres regímenes distintos. Utilizando el patrón de comportamiento Strategy, desarrolla una aplicación de consola en Java que calcule el sueldo neto de 3 empleados y el resumen de la planilla.

Datos que se calculan (valor hora = sueldo / 240):

- Pago por horas extra = horas extra × valor hora × 1.25
- Sueldo bruto = sueldo + pago por horas extra

Regímenes (cada uno calcula el sueldo neto de una forma):

- 1 = General: pensión = 13 % del bruto; renta = 8 % del exceso del bruto sobre 4000 (si el bruto es mayor a 4000; si no, 0). Neto = bruto − pensión − renta.
- 2 = Mype: pensión = 13 % del bruto; sin renta. Neto = bruto − pensión.
- 3 = Honorarios: no tiene horas extra, el bruto es solo el sueldo; retención = 8 % del sueldo si el sueldo es 3200 o más (si no, 0). Neto = sueldo − retención.

La solución deberá contemplar las siguientes clases:

- Interfaz SueldoStrategy, con el método double calcularNeto(double sueldo, int horasExtra).
- Clases RegimenGeneral, RegimenMype y RegimenHonorarios (implementan SueldoStrategy).
- Clase Empleado (contexto): tiene nombre, sueldo, horas extra y una SueldoStrategy; el método double calcularNeto() delega el cálculo en la estrategia.
- Clase Planilla: guarda la lista de empleados y tiene los métodos agregar(Empleado e), double calcularTotal() (suma de los netos) y Empleado mejorPagado() (el de mayor neto).
- Clase Main: pide por teclado los datos de 3 empleados (nombre, sueldo, horas extra y régimen 1, 2 o 3), crea la estrategia que corresponde, los agrega a la Planilla y muestra el neto de cada uno y el resumen.

Reglas importantes: ni Empleado ni Planilla deben tener if ni switch por régimen (la estrategia se elige solo en el Main). Muestra los resultados con 2 decimales y usa Locale.US (Scanner.useLocale(Locale.US) y String.format(Locale.US, "%.2f", valor)).

Datos de entrada y salida:

```
Nombre: Ana
Sueldo: 4800
Horas extra: 10
Régimen (1=General, 2=Mype, 3=Honorarios): 1

Nombre: Luis
Sueldo: 2400
Horas extra: 8
Régimen (1=General, 2=Mype, 3=Honorarios): 2

Nombre: Marta
Sueldo: 5000
Horas extra: 0
```

```
Régimen (1=General, 2=Mype, 3=Honorarios): 3

Ana - Neto: 4309.50
Luis - Neto: 2175.00
Marta - Neto: 4600.00
Total planilla: 11084.50
Mayor sueldo neto: Marta (4600.00)
```

Ejercicio 2: Patrón Observer (subasta en línea)

Una casa de subastas vende una laptop con precio base de S/ 1000.00. Cada vez que se acepta una nueva oferta, deben enterarse el historial, el cálculo de la comisión de la casa y los dos participantes. Utilizando el patrón de comportamiento Observer, desarrolla una aplicación de consola en Java que simule la subasta.

Reglas de la subasta:

- Una oferta es válida si es al menos 5 % mayor que la mejor oferta actual (al inicio, el precio base). Si no es válida, se muestra "Oferta rechazada: debe ser al menos S/ X" y no se notifica a nadie.
- La comisión de la casa es el 5 % de la oferta aceptada.
- El incremento (%) = (oferta nueva − oferta anterior) / oferta anterior × 100, donde la oferta anterior de la primera oferta es el precio base.

La solución deberá contemplar las siguientes clases:

- Interfaz Observador, con el método void actualizar(String postor, double oferta, double ofertaAnterior).
- Clase Subasta (sujeto): tiene el producto, la mejor oferta, el mejor postor y la lista de observadores; los métodos agregarObservador(Observador o), ofertar(String postor, double monto), que valida la oferta y notifica solo si es válida, y cerrar(), que muestra el ganador.
- Clase HistorialObservador: lleva la cuenta del número de oferta y muestra el incremento porcentual.
- Clase ComisionObservador: muestra la comisión de la casa.
- Clase ParticipanteObservador: tiene un nombre y un presupuesto máximo (se crean 2: Luis con 1250 y Rosa con 1500); muestra si su oferta lidera, si puede superar la oferta actual o si se retira porque la oferta supera su presupuesto.
- Clase Main: crea la subasta, registra los observadores (en este orden: Historial, Comisión, Luis y Rosa), pide por teclado 5 ofertas (postor y monto), y al final llama a cerrar().

Reglas importantes: el Main solo llama a subasta.ofertar(postor, monto); los cálculos y los mensajes (menos el de oferta rechazada y el de cierre, que son de la Subasta) los hacen los observadores. Usa Locale.US para leer y mostrar decimales.

Datos de entrada y salida:

```
Postor: Luis
Monto: 1100
Historial: oferta #1 de Luis S/ 1100.00 (+10.00%)
Comisión de la casa: S/ 55.00
Luis: tu oferta lidera
Rosa: puedes superar la oferta de S/ 1100.00

Postor: Rosa
Monto: 1150
Oferta rechazada: debe ser al menos S/ 1155.00
```

```
Postor: Rosa
Monto: 1200
Historial: oferta #2 de Rosa S/ 1200.00 (+9.09%)
Comisión de la casa: S/ 60.00
Luis: puedes superar la oferta de S/ 1200.00
Rosa: tu oferta lidera

Postor: Luis
Monto: 1300
Historial: oferta #3 de Luis S/ 1300.00 (+8.33%)
Comisión de la casa: S/ 65.00
Luis: tu oferta lidera
Rosa: puedes superar la oferta de S/ 1300.00

Postor: Rosa
Monto: 1400
Historial: oferta #4 de Rosa S/ 1400.00 (+7.69%)
Comisión de la casa: S/ 70.00
Luis: te retiras, la oferta supera tu presupuesto de S/ 1250.00
Rosa: tu oferta lidera

Subasta cerrada. Ganador: Rosa con S/ 1400.00
```

Ejercicio 3: Programación concurrente (cierre de caja de sucursales)

Una cadena de tiendas hace el cierre de caja de 4 sucursales. Cada cierre demora un tiempo (simulado con Thread.sleep) y hay solo 2 hilos disponibles en el servidor. Utilizando programación concurrente en Java, desarrolla una aplicación de consola que haga los cierres al mismo tiempo (como máximo 2 a la vez) y muestre el consolidado.

Datos de las sucursales:

- Lima: ventas 1200.50, 850.00 y 430.50; demora 3 segundos.
- Arequipa: ventas 900.00 y 1100.00; demora 2 segundos.
- Cusco: ventas 300.00, 450.00 y 250.00; demora 2 segundos.
- Trujillo: ventas 700.00, 650.00, 1000.00 y 150.00; demora 2 segundos.

Cálculos de cada cierre: total = suma de las ventas; neto sin IGV = total / 1.18.

La solución deberá contemplar las siguientes clases:

- Clase CierreSucursal (implementa Runnable): tiene el nombre, las ventas, los segundos de demora y los resultados total y neto. En run() muestra el mensaje de inicio (con el nombre del hilo), espera con Thread.sleep, calcula el total y el neto y muestra el mensaje de fin.
- Clase Main: crea las 4 tareas, usa un ExecutorService con un pool de 2 hilos (Executors.newFixedThreadPool(2)), envía las 4 tareas, cierra el pool (shutdown), espera a que terminen (awaitTermination), y muestra el total consolidado, el neto consolidado y el tiempo total.

Reglas importantes: no uses hilos de más (solo 2 en el pool); el Main suma los resultados solo después de que terminen todas las tareas; el tiempo total se mide con System.currentTimeMillis(). Si los cierres se hicieran uno tras otro tardarían 9 segundos; con 2 hilos deben tardar unos 5 segundos.

Datos de salida esperados (el orden de los dos primeros mensajes puede variar; los números de hilo pueden variar):

```
[pool-1-thread-1] Inicia cierre de Lima
[pool-1-thread-2] Inicia cierre de Arequipa
(a los 2 segundos)
[pool-1-thread-2] Termina Arequipa: total S/ 2000.00, neto S/ 1694.92
[pool-1-thread-2] Inicia cierre de Cusco
(a los 3 segundos)
[pool-1-thread-1] Termina Lima: total S/ 2481.00, neto S/ 2102.54
[pool-1-thread-1] Inicia cierre de Trujillo
(a los 4 segundos)
[pool-1-thread-2] Termina Cusco: total S/ 1000.00, neto S/ 847.46
(a los 5 segundos)
[pool-1-thread-1] Termina Trujillo: total S/ 2500.00, neto S/ 2118.64
Total consolidado: S/ 7981.00
Neto consolidado: S/ 6763.56
Tiempo total: 5 segundos (secuencial: 9 segundos)
```

Entregables:

- Un proyecto de NetBeans por ejercicio (ejercicio1, ejercicio2 y ejercicio3), cada uno comprimido en .zip, y un documento con la captura de la ejecución de cada uno.



Fecha límite de presentación: Determinados por el docente.
Integrantes: Determinados por el docente.